~/majibill/docs/13-scaling-playbook.md

```
$ cat 13-scaling-playbook.md
```

MAJIBILL PLATFORM · SCALE

Scaling Playbook

From a one-zone pilot to a Nairobi-scale utility and many counties: what scales, in what order, what triggers each step, and how to prove it before it matters.

```
# measure first, then scale the thing that is actually hot
# add cells before you shard
# a bill run is a batch job; a payment is a latency job
```

DOC	VERSION	DATE
MB-13	1.0 draft	08 Oct 2026

STATUS	AUDIENCE	CLASS
For review	Tech lead, backend, platform/DevOps, founders	Confidential

Prepared by 404HubSpot · Nakuru, Kenya · https://404-hub-spot.vercel.app

Working codename: MajiBill (Swahili: maji = water). Figures marked 'assumption' must be validated before commitment.

// Contents

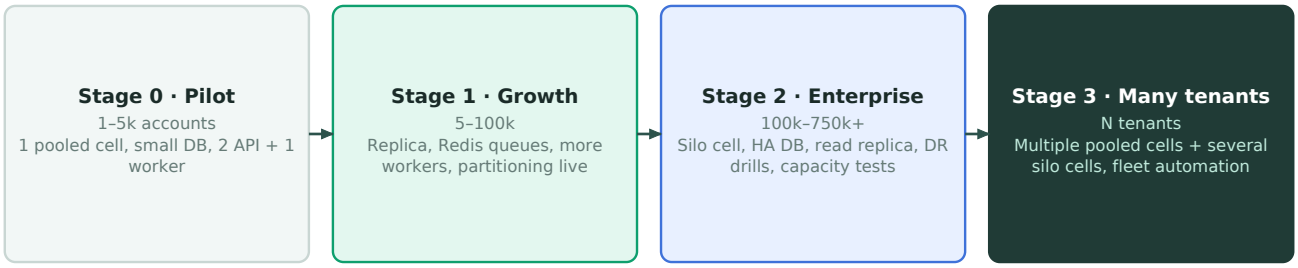
1. Scaling philosophy . . . 3
2. What actually scales . . . 3
3. Scaling triggers and thresholds . . . 4
4. Database scaling path . . . 4
5. Batch and event workloads . . . 5
6. Moving a tenant between cells . . . 6
7. Noisy-neighbour controls (pooled cells) . . . 6
8. Proving scale: load testing and capacity planning . . . 7
9. Organisational and operational scaling . . . 7

// 1. Scaling philosophy

MajiBill's load is unusual: a **monthly batch spike** (bill runs, SMS, reports), a **daily payment wave** around due dates, and a steady trickle of staff and field activity. Population is not the driver; **active billable connections, meters, and payment peaks** are. Scale in the cheapest order, and only on evidence.

1. **Measure before scaling.** Every step below has a metric trigger, not a calendar date.
2. **Fix the query before adding the server.** Most early wins are indexes, batching and caching.
3. **Scale out by cells, not by heroics.** Another cell is cheaper and safer than a bigger shared database (MB-01).
4. **Protect the money path.** Payment ingestion, posting and receipts get priority over reports and exports at every layer.
5. **Isolate noisy neighbours.** Per-tenant limits, queues and read replicas stop one tenant degrading others.
6. **Rehearse.** Load tests and restore drills at each stage, in a production-sized environment.

// 2. What actually scales



Each stage keeps the same software; only topology, sizing and automation change.

Fig 2.1 Capacity ladder.

Dimension	Driver	First lever	Next lever	Last resort
API and web	Concurrent staff and portal users	More replicas, CDN, caching	Read replica for list/report endpoints	Split reporting service
Bill run	Accounts per cycle	Route-parallel workers, batch writes	Dedicated worker pool, larger DB IOPS	Extract billing calculation service
Payments	Callbacks per second at peak	Go ingress replicas, inbox batching	Dedicated ingress pool per cell	Per-tenant ingress sharding
Database writes	Ledger, readings, audit rows	Indexes, batching, partitions, fast storage	Silo the heavy tenant to its own DB	Functional split (audit to its own cluster)
Database reads	Dashboards, reports, exports	Materialised views, caching	Read replicas, scheduled reports	Columnar analytics store with CDC
SMS / email	Messages per cycle	Queue rate limits, provider batching	Multiple providers, fallback routing	Dedicated notifier service
Storage	Photos, PDFs, audit	Lifecycle policies, compression	Tiered archive	—
Operations	Number of cells and tenants	IaC modules, dashboards per cell	Fleet tooling, automated tenant moves	Dedicated SRE function

// 3. Scaling triggers and thresholds

Metric	Healthy	Investigate	Act
API p95 latency (staff reads)	under 400 ms	400–800 ms for a week	Profile queries, add cache/replica
DB CPU (primary)	under 50% typical	above 65% sustained	Index/query review, then vertical step or silo
DB storage growth	linear	above plan by 20%	Retention, compression, partition detach
Connection usage (pooled)	under 60%	above 75%	Tune pooler, reduce chatty queries
Replication lag	under 5 s	above 30 s	Move heavy reports off replica; resize
Queue lag (payments)	under 10 s	above 60 s	Add workers; check DB locks
Bill run duration	within window	above 70% of window	More parallelism; tune batches
Ingress ack time (p99)	under 300 ms	above 800 ms	Add replicas; inspect DB insert path
Largest partition size	manageable vacuum	vacuum falling behind	Tune autovacuum; smaller partitions
Cost per account per month	stable	rising 2 months in a row	Right-size; audit log and storage

// 4. Database scaling path

1. **Design for it now:** tenant-first indexes, UUIDv7 keys, partitions by month on big append-only tables (MB-04).
2. **Tune:** query review with `pg_stat_statements`, covering indexes, avoiding N+1, bounded transactions, correct `fillfactor` and autovacuum on hot tables.
3. **Pool:** PgBouncer in transaction mode; cap connections per service.
4. **Offload reads:** read replica for reports and exports; materialised views for dashboards.
5. **Scale up:** larger instance and provisioned IOPS (fast, simple, buys months).
6. **Silo:** move the heaviest tenant to its own cell/DB (see section 6). This is the main horizontal strategy.
7. **Specialise:** audit and analytics to separate stores only if measurements demand it.
8. **Shard last:** by tenant across cells; cross-tenant reporting for partners uses aggregates replicated to the control plane.

> RISK **Do not scale by removing safety**
> Dropping RLS, constraints or the audit write to “go faster” is not a scaling strategy. Every optimisation must preserve tenant isolation, idempotency and the ledger invariants, and prove it with the same tests.

// 5. Batch and event workloads

5.1 Making the bill run scale

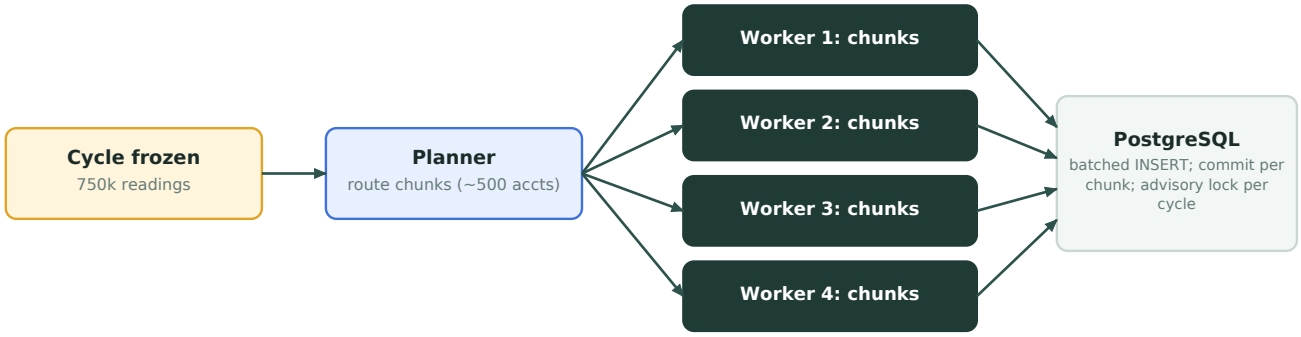


Fig 5.1 Parallel bill run: planner splits routes into idempotent chunks; workers commit per chunk; failures retry per chunk.

Technique	Effect
Chunking by route with idempotent keys	Resume after failure; linear speed-up with workers until DB limits
Set-based SQL for tariff lookups	Avoid per-account round trips; prefetch tariff versions per chunk
Batched inserts / `COPY`	10–100x faster than row-by-row for bill lines and ledger entries
Deferred, bounded index work	Keep write indexes minimal on hot partitions
Back-pressure	Workers pause if replication lag or queue depth crosses thresholds
Run windows	Schedule bill runs off the payment peak; payments always have priority

5.2 Payment peaks

- Go ingress is stateless and horizontally scalable; it only validates, persists to the inbox and acknowledges. Autoscale on request rate.
- Inbox processing workers scale on queue lag; per-account ordering is preserved by keying on account (or per-tenant ordering where required).
- Receipts SMS go to a separate queue so SMS provider slowness never delays posting.
- Design target: 100 tx/s burst per cell with headroom; expected Nairobi-scale peak is far lower (MB-01), so the real work is idempotency and reconciliation.

5.3 SMS at scale

- Per-tenant send rate and daily caps; provider-level batching and delivery-receipt processing.
- Two providers with automatic failover for large tenants.
- Cost control: segment counting before send, templates kept short, prefer portal/USSD links, wallet balances with alerts.

// 6. Moving a tenant between cells

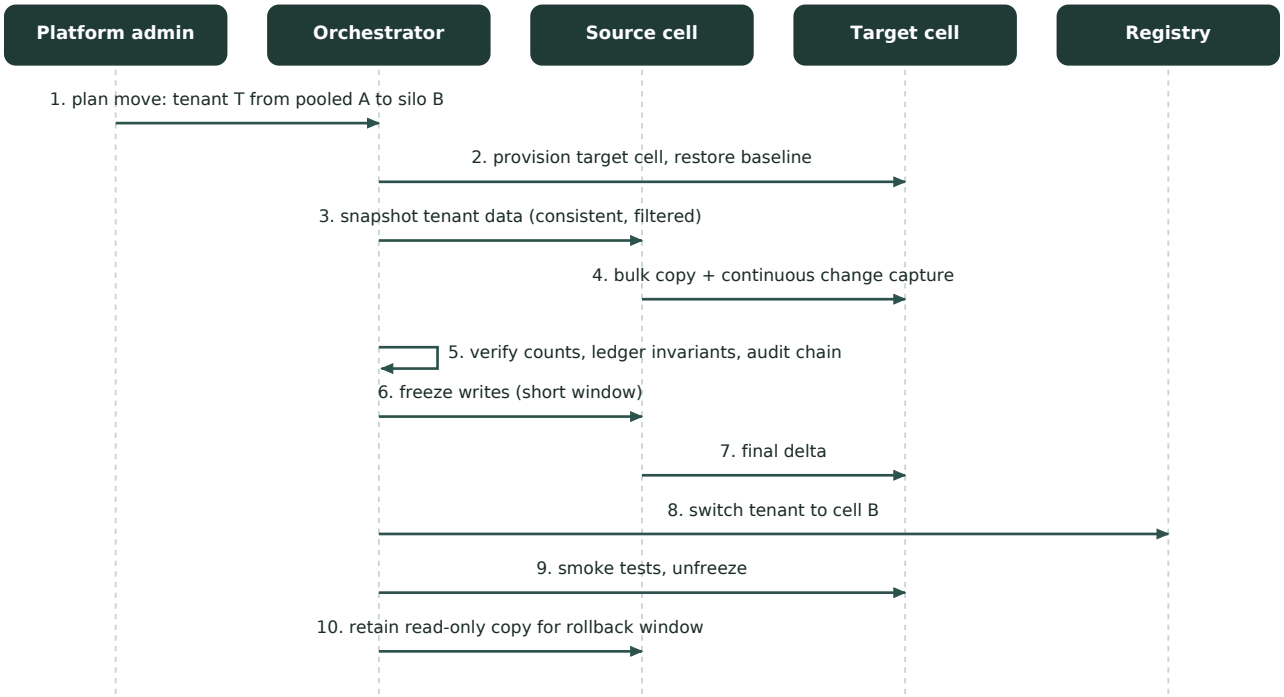


Fig 6.1 Tenant migration with a short write freeze and verification gates.

- Target downtime: minutes, scheduled in the tenant’s quiet window and communicated in advance.
- Rollback: registry switch back plus replay of changes made on the target; source copy retained until sign-off.
- Verification: row counts per table, ledger balance per account, audit chain continuity, sample bill recalculation; automated and attached to the change record.
- Same procedure serves silo promotion, region moves and the sovereign hand-off export.

// 7. Noisy-neighbour controls (pooled cells)

Resource	Control
API	Per-tenant and per-token rate limits; request cost classes (heavy reports limited)
Database	Statement timeouts, per-tenant concurrency caps on heavy jobs, replica routing for heavy reads
Queues	Per-tenant queues or weighted fair scheduling so one bill run cannot starve others
Storage and exports	Quotas, async exports with size limits, signed expiring links
Escalation	Automatic detection of a tenant consuming above fair share; candidate for silo promotion

// 8. Proving scale: load testing and capacity planning

Test	Scenario	Pass criteria
Bill run	750k accounts, realistic tariffs, in a production-sized silo cell	Under 2 h; no double bills; ledger balanced; payments unaffected
Payment burst	100 tx/s for 15 min with 5% duplicates and 2% unmatched	Zero lost, zero double-posted; ack p99 under 500 ms; lag under 60 s
Mixed day	Staff browsing + sync + reports + payments + SMS	p95 latency within SLO; replica lag under 30 s
Sync storm	5,000 readers uploading 100 readings each after a signal outage	No data loss; server stable; flagged items correct
Failure	Kill primary, saturate queue, delay SMS provider, corrupt a callback	Recovery within RTO; invariants hold; alerts fire
Soak	7 days at 60% load	No leaks; vacuum healthy; costs predictable

8.1 Cost of scale

At Nairobi scale the infrastructure cost per account is tiny compared with notifications. Using the planning ranges in MB-05 (about USD 2,500–3,500 per month for an enterprise cell) and 750,000 accounts, infrastructure is well under KES 1 per account per month (assuming roughly KES 130 per USD; verify), while a single SMS at the assumed KES 0.80 costs almost as much. **SMS policy and channel mix, not servers, dominate unit economics.**

- Tag all cloud resources by cell and tenant to track cost per account.
- Review cost per account monthly; target stable or falling as accounts grow.
- Prefer pooled cells for small tenants; give silos only to those who need them.

// 9. Organisational and operational scaling

Stage	Team and process change
Pilot	One squad; weekly release; founders on call; runbooks started
Growth	Two squads (core, field/integrations); platform engineer owns cells; on-call rotation of 3; monthly game day
Enterprise	Dedicated SRE/support function; incident commander role; change advisory for silo tenants; quarterly DR drills
Many tenants	Partner enablement team; tenant success managers; automated onboarding; status page; formal SLAs per tier

9.1 Multi-region and resilience roadmap

Step	When	Notes
Multi-AZ in one region	Pilot	Baseline for all cells
Encrypted cross-region or in-country backup copy	Pilot to Growth	Per tenant agreement and data-protection review
Warm standby in a second region for silo tenants	Enterprise, on request	Cost premium; RPO/RTO targets in MB-05
Active-active	Not planned	Complexity outweighs benefit for this workload; revisit only with strong demand

9.2 Scaling backlog (first items)

1. Instrument per-tenant metrics and cost tags from day one.
2. Build the bill-run planner and chunk worker with a synthetic data generator.
3. Create the 750k-account load-test dataset and k6 scenarios in staging.
4. Automate cell provisioning and tenant migration, rehearse once before any real move.
5. Write capacity runbooks: how to add workers, resize the DB, promote a tenant to silo.